1. INTRODUCTION TO DSA

* What is DSA ?

DSA stands for Data Structures and Algorithms.
Data Structures are ways to store and organize data in memory efficiently. Algorithms are step-by-step procedures or formulas to solve problems.

Remember

- Good DSA knowledge helps to write efficient, optimized and bug free code.
- DSA is the base for all advanced topics.

* Why is DSA important ?

→ Helps in solving complex problems in an efficient way.
→ Improves time and space efficiency of programs.
→ Required in almost every technical interview.
→ Forms the foundation for subjects like OS, DBMS, AI, ML, Networking etc.

Practice makes perfect !

* Why do companies ask DSA in interviews ?

→ To test problem solving and logical thinking.
→ To check how you approach a problem.
→ To evaluate your knowledge of fundamentals.
→ To see whether you can write optimized and clean code.

Interview Tip
Always analyze the problem and think of simple solution first, then optimize.

* Time Complexity

It measures the amount of time an algorithm takes to run as a function of input size (n).

* Space Complexity

It measures the amount of extra memory space used by an algorithm as a function of input size (n).

* Asymptotic Notations

① **Big O (O)** : Gives an upper bound. (Worst case complexity)
$f(n) = O(g(n))$ means there exist positive constants c and n_0 such that $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0$.

② **Omega (Ω)** : Gives a lower bound. (Best case complexity)
$f(n) = \Omega(g(n))$ means there exist positive constants c and n_0 such that $0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0$.

③ **Theta (Θ)** : Gives a tight bound. (Average case complexity)
$f(n) = \Theta(g(n))$ means there exist positive constants c_1, c_2 and n_0 such that $0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ for all $n \geq n_0$.

* Common Time Complexities

Complexity	Name	Example	When it occurs	Better than
$O(1)$	Constant	Accessing array element arr[i]	Same time for any input size	All
$O(\log n)$	Logarithmic	Binary Search	Divide input into half each time	$O(n)$, $O(n^2)$
$O(n)$	Linear	Linear Search	Single loop through n elements	$O(n \log n)$, $O(n^2)$
$O(n \log n)$	Linearithmic	Merge Sort, Heap Sort	Divide and conquer based	$O(n^2)$
$O(n^2)$	Quadratic	Nested Loops	Two nested loops	Worse

* Note : Lower time complexity is always better.

$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2)$

2. Arrays & Strings

• Arrays

An array is a collection of elements of the same data type stored in contiguous memory locations.
Elements are stored in linear manner and can be accessed using index.

Memory Representation :

Index	0	1	2	3	4
arr	10	20	30	40	50

arr[i] = element at i^{th} index
0 based indexing.

Common Operations :

- Access : O(1)
- Search : O(n)
- Insertion : O(n) (at end → O(1))
- Deletion : O(n)

Advantages :

✓ Random access is possible.
✓ Cache friendly.
✓ Easy to implement.

Disadvantages :

× Size is fixed.
× Insertion / Deletion in middle is costly.
× Requires contiguous memory.

5. Prefix Sum

Prefix sum helps to compute sum of subarrays in O(1) time after preprocessing.

Example :

arr = [3, 1, 2, 7, 4]
prefix[i] = sum of elements from index 0 to i
: prefix = [3, 4, 6, 13, 17]
Sum from i to j = prefix[j] − prefix[i−1]
→ (if i > 0)

Useful in range sum queries.

7. Kadane's Algorithm (Maximum Subarray Sum)

Finds the contiguous subarray with maximum sum.

Initialize : max_sum = −∞ , current_sum = 0

For each element x in array :

```
current_sum += x
max_sum = max(max_sum, current_sum)
if current_sum < 0 → current_sum = 0
```

Example : arr = [−2, 1, −3, 4, −1, 2, 1, −5, 4]

Answer : 6 (subarray [4, −1, 2, 1])

Time Complexity : O(n) Space Complexity : O(1)

2. Strings

A string is a sequence of characters.
In most languages, strings are treated as arrays of characters.

Example : s = "hello"

Index :	0	1	2	3	4
	h	e	l	l	o

Common Operations :

- Length : O(1)
- Access : O(1)
- Search : O(n)
- Concatenation : O(n)
- Substring : O(n)

Interview Tip
Always think about edge cases like empty string, single character, all same characters etc.

3. Two Pointer Technique

Use two pointers to traverse the data structure to solve problems efficiently.

Types :

1. Opposite Direction (start & end)
   Example : Pair with given sum in sorted array.
2. Same Direction (slow & fast together)
   Example : Remove duplicates from sorted array.

4. Sliding Window

A window of fixed or variable size is moved over the data structure to solve problems.

Types :

- Fixed Size Window
  Example : Maximum sum of subarray of size k.
- Variable Size Window
  Example : Smallest subarray with sum ≥ target.

Very useful for problems on arrays, strings, hashmaps, two pointers.

6. Examples of Problems

- Sliding Window : Longest substring without repeating characters.
- Two Pointers : 3Sum problem, Pair with target sum.
- Prefix Sum : Range sum queries, Subarray sum equals k.
- Kadane's Algo : Maximum subarray sum.

Key Takeaways

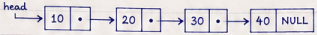
✓ Arrays provide random access in O(1).
✓ Strings are essentially arrays of characters.
✓ Two pointers reduce time complexity.
✓ Sliding window is used to optimize brute force.
✓ Prefix sums help in range queries.
✓ Kadane's algorithm helps in max subarray problems.

3. LINKED LISTS, STACK & QUEUE

1. Linked Lists

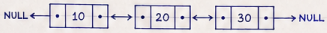
A linked list is a linear data structure in which elements are not stored at contiguous memory locations. Each element (node) contains data and a reference (link) to the next node.

a) Singly Linked List



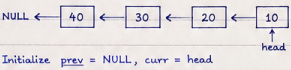
- Traversal is easy in forward direction only.

b) Doubly Linked List



- Each node has two links: prev and next.
- Useful for bidirectional traversal.

c) Reverse a Linked List (Iterative)



1. Initialize prev = NULL, curr = head
2. While curr != NULL
   next = curr->next
   curr->next = prev
   prev = curr
   curr = next
3. head = prev

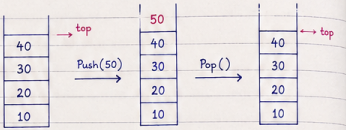
Interview Tip

Common questions:
- Find middle node
- Detect loop
- Merge two lists
- Remove N^{th} node from end

2. Stack (LIFO)

Stack follows Last In First Out principle.
Insertion and deletion happens from top.

Basic Operations :
- Push(x) : Add element x to top
- Pop() : Remove and return top element
- Peek() : Return top element
- isEmpty(): Check if stack is empty



Applications :
- Function call management (recursion)
- Undo operations in editors
- Expression evaluation, Parenthesis checking
- Backtracking, DFS in graphs

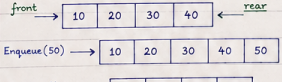
Example : Valid Parentheses

Given s = "(a+b)*(c-d)"
Scan left to right, push opening brackets.
On closing bracket, pop and match.
If stack empty at end → valid.

3. Queue (FIFO)

Queue follows First In First Out principle.
Insertion at rear and deletion from front.

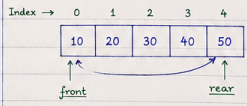
Basic Operations :
- Enqueue(x) : Add x at rear
- Dequeue() : Remove front element
- Front() : Return front element
- isEmpty() : Check if queue empty



Applications :
- CPU Scheduling, Printer Queue
- Breadth First Search (BFS)
- Handling real world waiting lines

4. Circular Queue

Last position is connected back to first position to make a circle. Efficient use of space.
Uses modulo operation: index = (index + 1) % size



Note

If front = (rear+1)%n
→ Queue is full
If front = rear
→ Queue is empty

Real Life Applications Summary

- Linked List → Dynamic memory, music playlist, undo-redo
- Stack → Browser back button, function calls, expression eval.
- Queue → Waiting lines, requests in web servers, BFS
- Circular Queue → CPU scheduling, buffering in streaming

Remember : Choose the right data structure based on the operation you need most frequently!
Think → Analyze → Code → Optimize

4. HASHING & TREES

1. Hashing

Hashing is a technique used to store and retrieve data in constant average time using a hash function. It is the backbone of HashMap, HashSet, etc.

a) Hash Function

A hash function takes a key and returns an index of the hash table.

Example : h(k) = k % m (m = size of table)

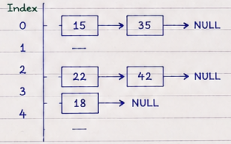
b) Hash Table Representation

Index →	0	1	2	3	4	5	6
	15	—	27	—	12	33	—

Each index can store one value (or a list of values in case of collision).

c) Collision Handling

- Separate Chaining (Using Linked List)



- Open Addressing (Linear Probing)

If collision occurs, try next index.

Example : insert key = 27, m = 7, h(k) = k%7

Index	0	1	2	3	4	5	6
Value	15	8	—	27	12	33	—

27%7 = 6

6 is filled, try next

6+1 = 7%7 = 0 (filled)

0+1 = 1 (placed at 1)

d) Applications

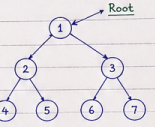
- Fast lookup (HashMap, HashSet)
- Frequency counting
- Caching
- Removing duplicates

Interview Tip

Always try to design the best hash function to minimize collisions. Good hash function => uniform distribution of keys.

2. Trees

A tree is a hierarchical data structure consisting of nodes. One node is marked as root and the remaining nodes are divided into disjoint subtrees of the root.



Tree Terminology

- Root : topmost node
- Parent : immediate upper node
- Child : immediate lower node
- Leaf : node with no children
- Height : longest path from root to leaf

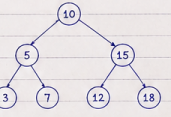
a) Binary Tree

Each node has at most two children (left and right child).

b) Binary Search Tree (BST)

For every node,

Left subtree < node < Right subtree



BST Operations

- Search : O(h)
- Insert : O(h)
- Delete : O(h)

h = height of tree

O(log n) in balanced BST, O(n) in worst case.

c) Tree Traversals

- Preorder (Root, Left, Right)
- Inorder (Left, Root, Right)
- Postorder (Left, Right, Root)
- Level Order (BFS)

Inorder

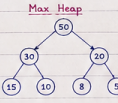
4, 2, 5, 1, 3

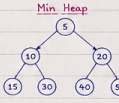
d) Heap (Binary Heap)

A complete binary tree that satisfies heap property.

- Max Heap : parent >= children
- Min Heap : parent <= children

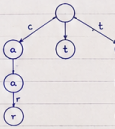
Used in Priority Queue, Heap Sort, Dijkstra.





e) Trie (Prefix Tree) – (Brief)

A trie is used to store strings so that operations like insert, search are efficient.



Useful for :

- Auto-complete
- Spell checker
- IP routing

5. GRAPHS

1. Introduction

A graph is a non-linear data structure consisting of a set of vertices (nodes) connected by edges. It is widely used to model real-world relationships like networks, maps, social connections, dependencies etc.

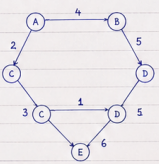
Terminology :

- Vertex (V) : A node in the graph.
- Edge (E) : Connection between two vertices.
- Undirected Graph : Edges have no direction.
- Directed Graph (Digraph) : Edges have direction.
- Weighted Graph : Each edge has a weight/cost.
- Degree : Number of edges connected to a vertex.

2. Types of Graphs

- Undirected Graph
- Directed Graph
- Weighted Graph
- Cyclic Graph : Contains at least one cycle.
- Acyclic Graph : No cycles.
- Connected Graph : Path between every pair.
- Disconnected Graph : Not connected.

Example Graph (Undirected, Weighted)



Note

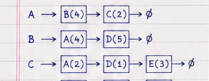
In a graph with V vertices :

- Max no. of edges (Undirected) = $\frac{V(V-1)}{2}$
- Max no. of edges (Directed) = $V(V-1)$

3. Graph Representations

a) Adjacency List

For each vertex, we store a list of all adjacent vertices.



Space Efficient for sparse graphs.

b) Adjacency Matrix

Use a V×V matrix. If there is an edge between i and j, store weight, else 0 (or ∞).

	A	B	C	D	E
A	0	4	2	0	0
B	4	0	0	5	0
C	2	0	0	1	3
D	0	5	1	0	6
E	0	0	3	6	0

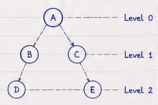
Efficient for dense graphs.

4. Traversals

a) BFS (Breadth First Search)

- Visits nodes level by level.
- Uses Queue.
- Time Complexity : O(V+E)

Example (Start from A)



BFS Order : A, B, C, D, E

b) DFS (Depth First Search)

- Goes as deep as possible before backtracking.
- Uses Stack / Recursion.
- Time Complexity : O(V+E)

Example (Start from A)

DFS Order : A, B, D, C, E

5. Shortest Path – Dijkstra's Algorithm

Finds shortest path from a source to all other vertices in a weighted graph with non-negative edge weights.

Steps :

1. Initialize distance[source] = 0, others = ∞
2. Put all vertices in a min-priority queue.
3. Extract vertex with minimum distance.
4. Update distances of its neighbors.
5. Repeat until queue becomes empty.

Time Complexity : O((V + E) log V)

Interview Tip

BFS is used for unweighted graphs (shortest path in terms of edges), while Dijkstra is used for weighted graphs.

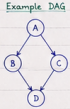
6. Topological Sort (For DAG)

A linear ordering of vertices in a Directed Acyclic Graph (DAG) such that for every directed edge u → v, u comes before v.

Method : Use DFS / Kahn's Algorithm (BFS based)

Applications : Task scheduling, build systems, course prerequisites.

Example DAG



One Topological Order

A, B, C, D

7. Union Find (Disjoint Set Union)

Data structure used to detect cycles and manage connected components.

- find(x) : returns representative of set containing x
- union(x, y) : merges sets of x and y

Used in : Kruskal's MST, Cycle detection, Connectivity.

Optimizations

- Path Compression
- Union by Rank / Size

8. Applications of Graphs

- GPS navigation (shortest path)
- Social networks (friend recommendations)
- Web crawling (search engines)
- Network routing
- Course prerequisites, task scheduling
- Minimum Spanning Tree in networks

Remember

Understand the problem, choose the right representation and algorithm.

6. Recursion & Dynamic Programming

1. Recursion

Recursion is a technique where a function calls itself to solve a smaller instance of the same problem. It must have a base case to stop further calls; otherwise it causes infinite recursion.

Key Points :

- Break problem into smaller subproblems.
- Solve base case directly.
- Combine results of recursive calls.

Example : Factorial of n (n! = n × (n-1)!)

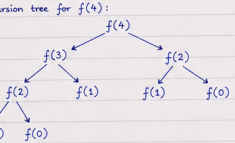
```
fact(n):
    if n == 0 or n == 1:
        return 1                    <-- base case
    return n * fact(n-1)            <-- recursive call
```

2. Recursion Tree (Example: Fibonacci)

Fibonacci : $f(n) = f(n-1) + f(n-2)$

$f(0) = 0,\ f(1) = 1$

Recursion tree for $f(4)$:



Time Complexity (naive recursion) : $O(2^n)$

3. Backtracking

Backtracking is an algorithmic technique for solving problems incrementally by trying to build a solution one step at a time and removing those solutions that fail to satisfy the constraints.

Examples : N-Queens, Sudoku Solver, Rat in Maze, Subset Sum Problem.

Interview Tip

- ✓ Always write the base case first.
- ✓ Draw recursion tree for better understanding.
- ✓ Check for overlapping subproblems → use DP.
- ✓ Backtracking is used when we need all possible solutions, not just one.

4. Dynamic Programming (DP)

DP is used to solve problems that have optimal substructure and overlapping subproblems.

It stores the results of subproblems to avoid recomputations.

Approaches :

- Memoization (Top – Down)
- Tabulation (Bottom – Up)

a) Memoization (Top – Down)

Use recursion + store results in a table (cache).

Reduces time complexity.

```
fib(n):
    if n <= 1: return n
    if dp[n] != -1: return dp[n] <-- already computed
    dp[n] = fib(n-1) + fib(n-2)
    return dp[n]
```

Remember

Initialize dp[] with -1 (or None) to indicate uncomputed state.

b) Tabulation (Bottom – Up)

Solve smaller subproblems first and build up to the final answer using a table.

Fibonacci using Tabulation :

```
fib(n):
    dp[0] = 0, dp[1] = 1
    for i from 2 to n:
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]
```

i	0	1	1	2	3	4	5	6
dp[i]	0	1	1	1	2	3	5	8

Time Complexity : $O(n)$

Space Complexity : $O(n)$

5. Classic DP Problems (Overview)

- Fibonacci Number → 1D DP
- 0/1 Knapsack → 2D DP
- Longest Common Subsequence (LCS) → 2D DP
- Longest Increasing Subsequence (LIS) → 1D DP
- Edit Distance → 2D DP

a) 0/1 Knapsack (Brief)

Given weights and values of n items and a capacity W, choose items to maximize value without exceeding W.

State : dp[i][w] = max value using first i items with capacity w.

Transition :

```
dp[i][w] = max( dp[i-1][w],
               value[i] + dp[i-1][w - wt[i]] )
If wt[i] <= w
else dp[i][w] = dp[i-1][w]
```

b) LCS (Brief)

Given two strings X and Y, find length of longest common subsequence.

State : dp[i][j] = LCS length for X[0..i-1] and Y[0..j-1]

Transition :

```
If X[i-1] == Y[j-1]
  dp[i][j] = 1 + dp[i-1][j-1]
else
  dp[i][j] = max( dp[i-1][j],
                  dp[i][j-1] )
```

Summary : Recursion is solving the problem. DP is optimizing the recursive solution. Use DP when you see : overlapping subproblems + optimal substructure.

7. Sorting & Searching

1. Sorting Algorithms

a) Bubble Sort

Repeatedly steps through the list, compares adjacent elements and swaps them if they are in wrong order.

Time Complexity : $O(n^2)$

Best Case : $O(n)$ (already sorted)

Stable : Yes

In-place : Yes

Example : arr = [5, 1, 4, 2, 8]

Pass 1: [1, 4, 2, 5, 8]

Pass 2: [1, 2, 4, 5, 8]

Pass 3: [1, 2, 4, 5, 8] (sorted)

b) Selection Sort

Finds the minimum element from unsorted part and places it at the beginning.

Time Complexity : $O(n^2)$

In-place : Yes Stable : No

Example : [64, 25, 12, 22, 11]

Pass 1 : [11, 25, 12, 22, 64]

Pass 2 : [11, 12, 25, 22, 64]

Pass 3 : [11, 12, 22, 25, 64]

Pass 4 : [11, 12, 22, 25, 64]

c) Insertion Sort

Builds the final sorted array one item at a time. Takes each element and inserts it in its correct position in the left sorted part.

Time Complexity : $O(n^2)$

Best Case : $O(n)$ (already sorted)

In-place : Yes Stable : Yes

Example : [5, 2, 4, 6, 1, 3]

i = 1 → [2, 5, 4, 6, 1, 3]

i = 2 → [2, 4, 5, 6, 1, 3]

i = 3 → [2, 4, 5, 6, 1, 3]

i = 4 → [1, 2, 4, 5, 6, 3]

i = 5 → [1, 2, 3, 4, 5, 6]

d) Merge Sort

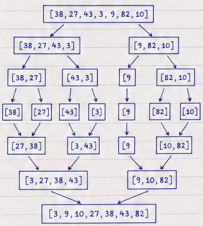
Divide the array into two halves, sort both halves and merge them.

Time Complexity : $O(n \log n)$

Space Complexity : $O(n)$

Stable : Yes In-place : No

Example : [38, 27, 43, 3, 9, 82, 10]



e) Quick Sort

Pick a pivot element. Place pivot at correct position by partitioning and recursively sort the subarrays.

Time Complexity :

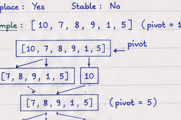
Best/Average : $O(n \log n)$

Worst Case : $O(n^2)$ (already sorted)

Space Complexity : $O(\log n)$ (avg)

In-place : Yes Stable : No

Example : [10, 7, 8, 9, 1, 5] (pivot = 10)



Sorted → [1, 5, 7, 8, 9, 10]

2. Searching Algorithms

a) Linear Search

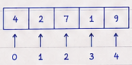
Check each element one by one until the target element is found.

Time Complexity : $O(n)$

Space Complexity : $O(1)$

Use Case : Unsorted arrays, small datasets.

Example : arr = [4, 2, 7, 1, 9], target = 7



Found at index 2.

b) Binary Search

Works on sorted arrays. Repeatedly divides the search interval in half.

Time Complexity : $O(\log n)$

Space Complexity : $O(1)$ (iterative)

Example : arr = [1, 3, 5, 7, 9, 11, 13]

target = 7

low = 0, high = 6 → mid = 3 → arr[3] = 7

Found!

Otherwise adjust the range accordingly.

When to Use What?

- Bubble Sort → small data, teaching
- Selection Sort → minimal swaps
- Insertion Sort → almost sorted data
- Merge Sort → stable, large data
- Quick Sort → in-place, fast in practice
- Linear Search → unsorted data
- Binary Search → sorted data

3. Time Complexity Comparison

Algorithm	Best Case	Average Case	Worst Case	Space Complexity	Stable	In-place
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	No
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Yes
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$	–	Yes
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$	–	Yes

Interview Tip

Always mention :

- ✓ Time Complexity
- ✓ Space Complexity
- ✓ Whether it is stable
- ✓ Whether it is in-place
- ✓ When to use it

Remember : Understand the intuition behind each algorithm. Dry run with example. Analyze edge cases.

8. Interview Patterns

Recognizing patterns helps to choose the right approach quickly. Below are common patterns frequently asked in coding interviews along with key idea and example problems.

1. Sliding Window

Maintain a window (subarray/substring) and move its start/end to solve problems efficiently.

Example Problems:

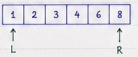
- Maximum Sum Subarray of size K
- Longest Substring Without Repeating Characters

2. Two Pointers

Use two pointers to traverse the data structure from different sides or with different speeds.

Example Problems:

- Pair Sum in Sorted Array
- 3Sum / 4Sum



3. Fast & Slow Pointer

Use two pointers moving at different speeds.

Useful for cycle detection and finding middle.

Example Problems:

- Linked List Cycle
- Middle of Linked List

4. Binary Search Pattern

Apply binary search on sorted data or answer space to reduce search space by half.

Example Problems:

- Search in Rotated Sorted Array
- Find Minimum in Sorted Array

5. Greedy Algorithm

Make the locally optimal choice at each step hoping to find the global optimum.

Example Problems:

- Activity Selection
- Fractional Knapsack

Greedy Choice Property

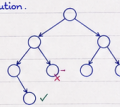
A globally optimal solution can be reached by making a series of locally optimal choices.

6. Backtracking

Build solutions step by step and backtrack when a choice doesn't lead to a valid solution.

Example Problems:

- N-Queens
- Sudoku Solver



7. Monotonic Stack

Keep elements in stack in increasing or decreasing order. Helps to find the next greater/smaller element.

Example Problems:

- Next Greater Element
- Largest Rectangle in Histogram

8. Prefix Sum

Precompute prefix sums to answer range sum queries in O(1) time.

Example Problems:

- Range Sum Query - Immutable
- Subarray Sum Equals K

arr	2	4	1	3	5
prefix	2	6	7	10	15

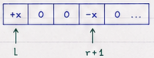
sum(l..r) = prefix[r] - prefix[l-1]

9. Difference Array

Efficient for range update queries. Mark changes at the start and end+1 of the range.

Example Problems:

- Range Addition
- Car Pooling

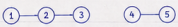


10. Union Find (Disjoint Set Union)

Maintain disjoint sets and support union and find operations efficiently.

Example Problems:

- Number of Islands II
- Redundant Connection



Union: connect two sets

Find: find representative (root)

Pattern Selection Tips

- ✓ Understand what the problem is asking.
- ✓ Identify constraints (time/space).
- ✓ Match problem characteristics to patterns.
- ✓ Practice multiple problems for each pattern.

Remember

- ✓ Most problems can be solved using a combination of patterns.
- ✓ Don't memorize code, understand the idea behind the pattern.
- ✓ Break the problem → choose pattern → implement → optimize.

★ Practice is the key to pattern recognition and speed in interviews. ★

9. Advanced Tree Concepts

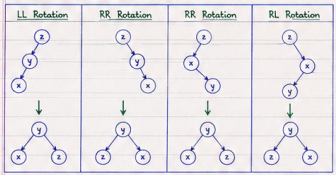
1. AVL Trees (Balanced BST)

An AVL tree is a self-balancing BST in which the height difference (balance factor) of left and right subtrees of every node is at most 1.

Balance Factor = height(left) − height(right)

⇒ Balance Factor ∈ {-1, 0, +1}

Rotations (to restore balance)



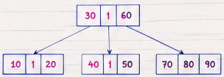
Key Points :
- Search, Insert, Delete : O(log n)
- Height of AVL tree : O(log n)

2. B-Trees (Balanced Multi-way Search Trees)

A B-Tree of order m is a balanced tree that keeps data in sorted order and allows more than two children per node. It is widely used in databases and file systems.

Properties of B-Tree (order m)
1. Every node has at most m children.
2. Every non-root node has at least ⌈m/2⌉ children.
3. Root has at least 2 children (unless it is a leaf).
4. All leaves appear at the same level.
5. Keys in each node are sorted.

Example : B-Tree of order 4



Time Complexity
- Search : $O(\log_m n)$
- Insert : $O(\log_m n)$
- Delete : $O(\log_m n)$

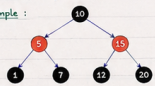
Applications :
- Database indexing
- File systems (e.g., NTFS)
- Disk storage systems

3. Red-Black Trees

A Red-Black Tree is a self-balancing BST where each node is colored Red or Black and the following properties hold.

Properties
1. Every node is either Red or Black.
2. Root is Black.
3. All leaves (NULL nodes) are Black.
4. No two consecutive Red nodes.
5. For each node, all paths from the node to its descendant NULL nodes contain the same number of Black nodes (Black Height).

Example :

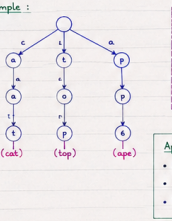


Key Points
- Height : O(log n)
- Search / Insert / Delete : O(log n)
- More flexible than AVL but harder to implement.

4. Trie (Prefix Tree)

A Trie is a tree used to store strings so that operations like insert, search and prefix queries are efficient.

Example :



Operations & Complexity
- Insert : O(L)
- Search : O(L)
- StartsWith(prefix) : O(L)

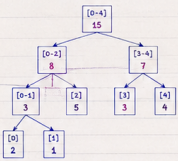
Where L = length of string

Applications :
- Auto-complete
- Spell checking
- IP routing (Longest prefix match)

5. Segment Tree

A Segment Tree is used for range queries and updates on an array. It is a complete binary tree.

Example : Array A = [2, 1, 5, 3, 4]



Operations
- Range Sum Query : O(log n)
- Point Update : O(log n)

Applications :
- Range Sum / Min / Max
- Count of numbers
- Competitive programming

6. Heavy-Light Decomposition (HLD)

A technique to decompose a tree into paths so that path queries can be answered using segment trees.

Idea :
- Select heavy child (largest subtree).
- Keep heavy edges in the same path.
- Break light edges and continue.

Complexity
- Update / Query on path : $O(\log^2 n)$ (with segment tree) or $O(\log n)$ (with advanced data structures)

Applications :
- Path queries on trees
- Subtree queries
- Competitive programming

★ Trees are powerful tools → Choosing the right structure makes a big difference! ★